# DSC 40B - Homework 07

Due: Tuesday, November 17

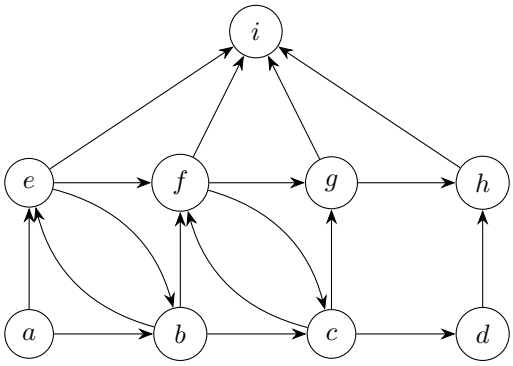
**Instructions:**

- Write your solutions to the following problems **by hand**, either on paper that you scan or using a tablet. Typed solutions will not be accepted for credit!
  - Code is an exception. You do not need to handwrite code; you can instead include it (and any plots it makes) as a screenshot.
- Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer.
- Homework problems are graded pass/fail on effort, not correctness.
- Homeworks are due via Gradescope at 11:59 PM.

**Problem 1.** ($\frac{1}{2}$ credit)

In lecture, we saw that breadth-first search (BFS) can be used to find the shortest paths from a source node to all other nodes in an unweighted graph. Our implementation of BFS returned two dictionaries: `distance`, containing the shortest path distance from the source node to each node in the graph, and `predecessor`, containing the BFS predecessor of each node.

Consider a breadth-first search on the graph shown below, starting with node $a$.



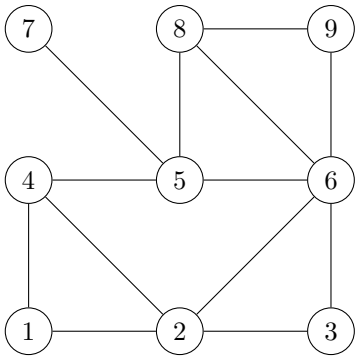

Write down the `distance` and `predecessor` dictionaries returned by BFS on this graph.

**Note:** you should adopt the convention that `.neighbors()` returns the neighbors of a node in **ascending alphabetical order**.

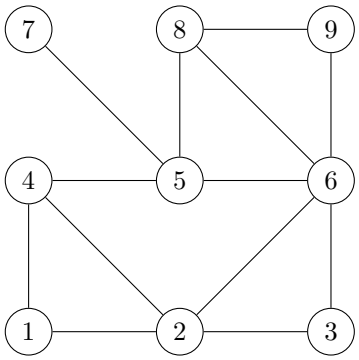
**Problem 2.** (1 credit)

For the following parts, recall that $(u, v)$ is a *tree edge* if node $v$ is discovered while visiting node $u$ during a breadth-first or depth-first search. Assume the convention that a node's neighbors are produced in ascending order by label. You do not need to show your work for this problem.

**a)** Suppose a breadth-first search is performed on the graph below, starting at node 1. Mark every BFS tree edge with a bold arrow emanating from the predecessor.




**b)** Suppose a depth-first search is performed on the graph below, starting at node 1. Mark every DFS tree edge with a bold arrow emanating from the predecessor.



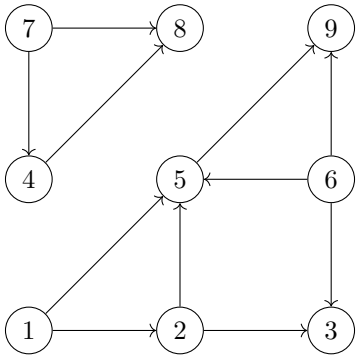

**c)** Fill in the table below so that it contains the start and finish times of each node after a DFS is performed on the above graph using node 1 as the source. Begin your start times with 1.

| Node | Start | Finish |
|---|---|---|
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |
| 6 | | |
| 7 | | |
| 8 | | |
| 9 | | |

**Problem 3.** ($\frac{1}{2}$ credit)

Give two different valid topological sorts of the nodes of the following graph. (There may be many equally-correct topological sorts.)

You do not need to show your work for this problem.

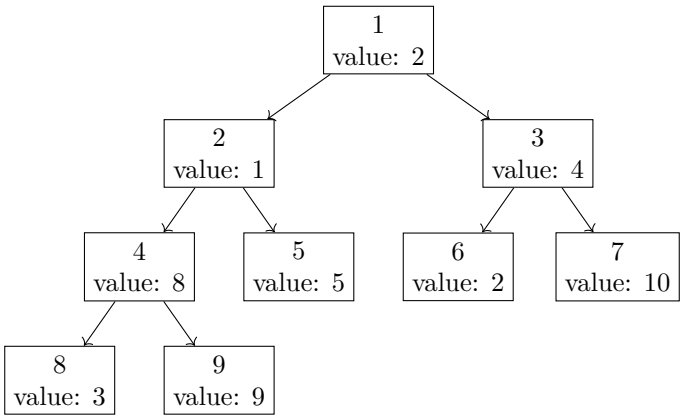



**Programming Problem 1.** ($\frac{1}{2}$ credit)

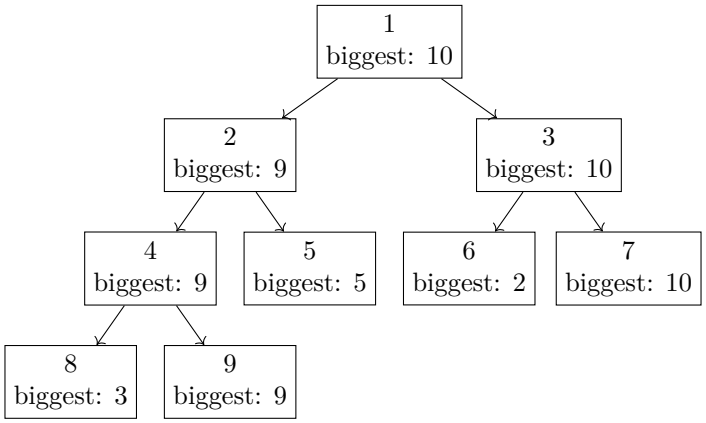
You are given a directed graph representing a tree and a dictionary `value` which contains a value for each node. Define the *biggest descendent value* of a node $u$ to be the largest value of any node which is a descendent

of $u$ in the tree (for this problem, you should consider $u$ to be a descendent of itself).

For instance, given the following tree where each node's label is replaced by its value:




The *biggest descendent value* for each node is:




In a file named `biggest_descendent.py`, write a function `biggest_descendent(graph, root, value)` which accepts the graph, the label of the root node, and the dictionary of values and returns a dictionary mapping each node in the graph to its biggest descendent value.

The input graph will be an instance of `dsc40graph.DirectedGraph`, with each edge directed from a parent to its child. Your function should take $O(|V|)$ time. The autograder will check this by running your code on a large tree.

As this is a programming problem, submit your `biggest_descendent.py` file to the Gradescope autograder assignment named "Homework 07 - Programming Problem 01", which has the same due date as the written homework. You don't need to write your code by hand or include it (or a screenshot of it) in your written homework; submitting it to the autograder is enough.

Example:

```
>>> edges = [(1, 2), (1, 3), (2, 4), (2, 5), (4, 8), (4, 9), (3, 6), (3, 7)]
>>> g = dsc40graph.DirectedGraph()
>>> for edge in edges: g.add_edge(*edge)
>>> value = {1: 2, 2: 1, 3: 4, 4: 8, 5: 5, 6: 2, 7: 10, 8:3, 9: 9}
>>> biggest_descendent(g, 1, value)
{1: 10, 2: 9, 3: 10, 4: 9, 5: 5, 6: 2, 7: 10, 8: 3, 9: 9}
```

**Programming Problem 2.** ($\frac{1}{2}$ credit)

Suppose we are given a weighted, undirected graph $G = (V, E, \omega)$ in which the edge weights represent *similarity*; for example, the similarity between two users in a social network. Given a number $\lambda$, which we call the *level*, we will say that the *clusters* of $G$ are the connected components of the graph after all edges whose weight is less than $\lambda$ have been removed. There are other ways of defining the clusters of a weighted graph, but this is one natural way.

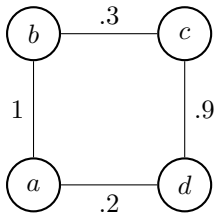
In `cluster.py`, write a function `cluster(graph, weights, level)` which computes the clusters of a weighted graph. Here, `graph` is an instance of `dsc40graph.UndirectedGraph`, `weights(u, v)` is a function returning the weight of edge $(u, v)$, and `level` is a number representing the level at which to find the clusters. Its return value should be a `frozenset`[1] containing `frozenset`s; the inner `frozenset`s should contain the nodes in a cluster.

Your code should not modify the graph in any way and it should not create a copy of the graph. It should run in $\Theta(V + E)$ time.

Note that `weights` is a function that will be constructed by us and passed to your function; you will not need to create it (except in your own tests).

As this is a programming problem, submit your `cluster.py` file to the Gradescope autograder assignment named "Homework 07 - Programming Problem 02", which has the same due date as the written homework. You don't need to write your code by hand or include it (or a screenshot of it) in your written homework; submitting it to the autograder is enough.

For example, given the following graph:




The output when run with a level of 0.4 should be:

```
>>> def weights(x, y):
...     x, y = (x, y) if x < y else (y, x)
...     return {("a", "b"): 1, ("b", "c"): .3, ("c", "d"): .9, ("a", "d"): .2}[(x, y)]
>>> cluster.cluster(graph, weights, 0.4)
frozenset([frozenset(['a', 'b']), frozenset(['c', 'd'])])
```

Note: you might see curly braces instead of square brackets in the output – that's OK. Different versions of Python print frozensets differently.

[1] We use `frozenset`s here because clusters are sets, but `set`s cannot be elements within other `set`s since they are not hashable.